



Harish's Master FDE / AI Automation Roadmap

As of @October 4, 2026 · Built from your resume and merged with "RPA + FDE" and "Forward Deployed Engineer / AI Automation Engineer"

How this roadmap works

Six phases over about 24 weeks. Each phase ends with one portfolio project and an **apply gate**: the remote roles you can apply for with what you've built so far. Keep applying from Gate A onward; each gate adds better-paid titles. Remote first; relocation (Hyderabad or Bengaluru) is the fallback after Gate D.

- **Time:** 2–3 hours on weekdays, 3–4 hours on Saturday building, Sunday for revision and applications.
- **Data rule:** every project uses synthetic data you generate yourself (Faker, reportlab). No client data, no client names, nothing that breaks company policy.
- **Resume rule:** don't list individual client projects. Keep the domain groupings your resume already uses (Finance, Sales, Supply Chain, HR & Compliance) and add these portfolio projects in a separate "Projects" section with GitHub links.

Phase	Weeks	Project	Apply gate	Rough target
0 · SQL + Python recap	1–4	Finance Reporting Automation Engine	A: Reporting / BI automation, Python automation	13–18 LPA
1 · Microsoft automation layer	5–7	GST Reconciliation in the Cloud	B: Intelligent automation, Power Platform	15–20 LPA
2 · Engineering foundations	8–11	Payroll Bank Mandate Validation API	C: Python automation,	15–22 LPA

Phase	Weeks	Project	Apply gate	Rough target
			integration engineer	
3 · Modern data stack (properly)	12–14	Sales & Inventory Analytics Warehouse	C+: Analytics engineer, data engineer	15–22 LPA
4 · AI engineering	15–19	Invoice-to-Ledger AI + Ask-Your-Finance-Data	D: AI automation, applied AI	18–25 LPA
5 · Agents and production	20–24	AI Data Operations Agent (flagship)	E: FDE, AI deployment, solutions engineer AI	20–30+ LPA

Targets are rough planning numbers, not guarantees. A genuine FDE role with production AI work at 15–20 is worth taking: it sets up 30+ on the next move.

Phase 0 · SQL + Python recap (weeks 1–4)

Weeks 1–2.5: recap with hands-on practice in PostgreSQL and VS Code. Weeks 2.5–4: build Project 0. Do 30 minutes of interview-style practice daily without AI.

SQL recap (PostgreSQL)

Setup

- ☐ Install PostgreSQL + DBeaver (or pgAdmin); create a practice database
- ☐ Load a sample dataset (or the synthetic finance data from Project 0, step 1)

Fundamentals

- ☐ SELECT, WHERE, ORDER BY, LIMIT/OFFSET, DISTINCT
- ☐ Operators: IN, BETWEEN, LIKE, ILIKE, comparison and logical operators
- ☐ NULL handling: IS NULL, COALESCE, NULLIF, NULLs in comparisons and aggregates
- ☐ CASE WHEN (simple and searched)

☐ Column and table aliases; CAST and  type conversion; ROUND, numeric precision

Aggregation

- ☐ COUNT, COUNT DISTINCT, SUM, AVG, MIN, MAX
- ☐ GROUP BY on multiple columns; HAVING vs WHERE
- ☐ Conditional aggregation: SUM(CASE WHEN ...), Postgres FILTER clause
- ☐ GROUPING SETS, ROLLUP, CUBE (subtotals like an Excel pivot)

Joins

- ☐ INNER, LEFT, RIGHT, FULL OUTER
- ☐ Self join (e.g. employee–manager, account–parent account)
- ☐ CROSS JOIN for scaffolding (every entity × every month)
- ☐ Anti-join: LEFT JOIN ... WHERE key IS NULL, and NOT EXISTS
- ☐ Semi-join: EXISTS
- ☐ Join pitfalls: duplicate fan-out, NULL keys, many-to-many

Subqueries and CTEs

- ☐ Scalar, IN and correlated subqueries
- ☐ Chained CTEs for step-by-step logic
- ☐ Recursive CTE: hierarchy (chart of accounts) and date series

Window functions

- ☐ ROW_NUMBER, RANK, DENSE_RANK; deduplication with ROW_NUMBER
- ☐ PARTITION BY + ORDER BY
- ☐ LAG / LEAD: month-on-month change
- ☐ Running totals and YTD: SUM() OVER (ORDER BY ...)
- ☐ Moving averages with ROWS BETWEEN
- ☐ FIRST_VALUE, LAST_VALUE, NTILE; percent of total

Dates and strings

- ☐ DATE_TRUNC, EXTRACT, intervals, date differences, AGE

- ☐ Fiscal year and fiscal month logic (April–March)
- ☐ generate_series to build a calendar / dim_date table
- ☐ TRIM, UPPER/LOWER, SUBSTRING, SPLIT_PART, CONCAT, REPLACE
- ☐ Regex: REGEXP_REPLACE,  matching (e.g. validate GSTIN or IFSC formats)

Set operations and reconciliation

- ☐ UNION vs UNION ALL, INTERSECT, EXCEPT
- ☐ Reconciliation queries: source vs target totals, missing records with EXCEPT, mismatched amounts

Changing data

- ☐ INSERT, INSERT ... SELECT, UPDATE with a join, DELETE
- ☐ Upsert: INSERT ... ON CONFLICT DO UPDATE; MERGE
- ☐ Transactions: BEGIN, COMMIT, ROLLBACK

Modelling and DDL

- ☐ CREATE TABLE, data types, PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL, DEFAULT
- ☐ Normalisation: 1NF, 2NF, 3NF
- ☐ Star schema: facts vs dimensions, surrogate keys, grain
- ☐ Slowly changing dimensions: type 1 and type 2
- ☐ Audit columns: created_at, updated_at, loaded_by, source_file
- ☐ Views and materialized views (REFRESH MATERIALIZED VIEW)

Performance

- ☐ Indexes: B-tree, composite, when an index is not used
- ☐ EXPLAIN and EXPLAIN ANALYZE; reading a query plan
- ☐ Sargable filters; avoid SELECT * in production queries
- ☐ Partitioning (concept)

Data quality checks in SQL

- ☐ Duplicate detection, null-rate checks, orphan records (referential integrity)

- ☐ Range and outlier checks; row-count and total reconciliation between layers

Practice

- ☐ LeetCode SQL 50 study plan or DataLemur: 40 problems, medium level
- ☐ Explain every answer out loud as if in an interview

Python recap

Core language

- ☐ Types: int, float, str, bool, None; f-strings and string methods
- ☐ Lists, tuples, dicts, sets; when to use which
- ☐ Comprehensions: list, dict, set; nested comprehensions
- ☐ Control flow; enumerate, zip, unpacking
- ☐ Functions: positional and keyword args, defaults, *args*, *kwargs, return values
- ☐ lambda, sorted with key, map, filter
- ☐ Modules, packages, imports; `if __name__ == "__main__":`

Files and folders

- ☐ pathlib for paths; glob patterns; `with open()`
- ☐ csv and json modules; reading many files from a folder
- ☐ shutil: move, copy, archive processed files; zip files

Error handling

- ☐ try / except / else / finally
- ☐ Raising errors; custom exception classes (e.g. ValidationError, MissingColumnError)
- ☐ Retryable vs non-retryable errors; simple retry loop with backoff

Classes

- ☐ Classes, **init**, methods, attributes
- ☐ dataclasses
- ☐ Composition vs inheritance; @property

- ☐ Reading class-based code in real repos

Type hints and validation

- ☐ Type hints: list[str], dict[str, float], Optional, Union, Literal
- ☐ TypedDict
- ☐ Pydantic BaseModel: validation, field constraints, custom validators

Logging and config

- ☐ logging module: levels, format, file handler, one logger per module
- ☐ Environment variables, `.env` with python-dotenv
- ☐ Config files (YAML or TOML)

Environment and packaging

- ☐ venv, pip, requirements.txt; try uv
- ☐ Project structure: app/, tests/, data/, logs/, config/
- ☐ Command-line scripts with argparse or Typer
- ☐ Packaging an exe with PyInstaller (you've done this; know how it works)

Pandas (data work)

- ☐ read_excel (multiple sheets, header rows, usecols), read_csv; dtypes on read
- ☐ Selection: loc, iloc, boolean masks, query
- ☐ groupby + agg (named aggregations)
- ☐ merge (inner/left, validate="one_to_one"), concat
- ☐ pivot_table and melt
- ☐ Dates: to_datetime, .dt accessor, resample, period
- ☐ .str methods for cleaning text
- ☐ Missing data, duplicates, data types, outliers
- ☐ Vectorisation vs apply; processing large files in chunks
- ☐ Writing Excel with openpyxl: formats, column widths, formulas, multiple sheets

Databases from Python

- ☐ SQLAlchemy engine + psycopg
- ☐ `pd.read_sql` and `df.to_sql`; bulk loads
- ☐ Parameterised queries (never build SQL with f-strings: SQL injection)
- ☐ Transactions from Python

APIs from Python

- ☐ requests or httpx: GET, POST, headers, query params, JSON bodies
- ☐ Auth: API keys, bearer tokens
- ☐ Pagination, timeouts, retries with backoff, handling 4xx vs 5xx

Testing and tooling

- ☐ pytest: test functions, fixtures, parametrize, testing pandas transforms
- ☐ Ruff for lint and formatting
- ☐ VS Code debugger and breakpoints
- ☐ Git: init, add, commit, branch, merge, push, pull requests, .gitignore

Scheduling

- ☐ Windows Task Scheduler and cron basics

Practice

- ☐ 30 interview-style problems: strings, hash maps, sorting, two pointers, parsing CSV and logs
- ☐ Solve without AI, then compare with an AI solution

Project 0 · Finance Reporting Automation Engine

Your day job, rebuilt cleanly and publicly: monthly P&L with currency restatement, budget vs actual, and exception reporting.

Step 1: Generate synthetic data

- ☐ Python + Faker: GL transactions for 3 companies, 4 business units, 12 months, 3 currencies
- ☐ Masters in Excel: chart of accounts (with parent accounts), entity master, BU master
- ☐ Budget file (Excel) and monthly FX rates file (CSV)

- ☐ Deliberately inject errors: unknown accounts, missing FX rates, unbalanced journals, duplicates, wrong date formats

Step 2: Design the database (PostgreSQL)

- ☐ Tables: dim_account, dim_entity, dim_bu, dim_date, dim_currency, fx_rates, fact_gl, fact_budget, load_log, exceptions
- ☐ DDL with primary/foreign keys, CHECK constraints, audit columns
- ☐ dim_date built with generate_series, including fiscal year (April–March)

Step 3: Ingestion with validation (Python)

- ☐ Watch an input/ folder for monthly files
- ☐ Schema checks: required columns, data types, sheet names
- ☐ Business rules: debits = credits per journal, account exists in master, FX rate exists for currency and month, no duplicate journal lines
- ☐ Valid rows → upsert into fact_gl; invalid rows → exceptions table + exception_report.xlsx
- ☐ Move processed files to archive/ and record each run in load_log

Step 4: Transformations (SQL)

- ☐ View: P&L by entity, BU and month using the account hierarchy (recursive CTE)
- ☐ Currency restatement to reporting currency at actual and constant rates
- ☐ Budget vs actual variance and variance %
- ☐ YTD and month-on-month with window functions

Step 5: Outputs

- ☐ Formatted Excel MIS pack generated with openpyxl (summary sheet + detail sheets)
- ☐ Power BI report connected to the views

Step 6: Engineering

- ☐ Logging to logs/, config in .env, one command: `python run.py --month 2026-09`
- ☐ pytest tests for every validation rule
- ☐ Git history with meaningful commits

Step 7: Publish

- ☐ README: problem, architecture diagram, data model, validation rules, before/after (manual hours vs runtime), screenshots
- ☐ 3-minute walkthrough video

Resume line: *Built a Python + PostgreSQL finance reporting engine that validates multi-entity GL data, restates currencies, and produces P&L and budget-variance MIS packs with automated exception reporting.*

Gate A · Apply from week 4

Titles: Reporting Automation Engineer, BI Automation Engineer, Python Automation Engineer, Analytics Engineer, Finance Data Analyst (automation)

- ☐ Update resume headline to "Data & Process Automation Engineer | Python | SQL | Power BI"
 - ☐ 5 targeted remote applications a week from now on
-

Phase 1 · Microsoft automation layer (weeks 5–7)

Learn

- ☐ Power Automate cloud flows: triggers, actions, conditions, variables, expressions, Apply to each, scheduled flows
- ☐ SharePoint, OneDrive, Outlook, Teams, Excel connectors; approvals; adaptive cards in Teams
- ☐ Error handling: Configure run after, scopes (try/catch pattern), retry policies
- ☐ Licensing: check which connectors (e.g. HTTP) need premium on your plan; use Microsoft's developer plan if available to you
- ☐ Azure basics: resource groups, Storage, Key Vault, Azure Functions in Python (HTTP trigger)
- ☐ Copilot Studio: a small agent that triggers a flow

Project 1 · GST Reconciliation in the Cloud

Purchase register vs supplier-reported data (GSTR-2B style), automated end to end in the Microsoft stack.

Step 1: Synthetic data

☐ Purchase register (Excel) and supplier-reported invoices (Excel), generated with Faker

☐ Inject cases: exact match, amount mismatch, invoice-number typo, missing on either side, wrong GSTIN

Step 2: SharePoint setup

☐ Document library with folders Incoming / Processed / Exceptions

☐ A SharePoint list to log each run (file, status, counts, timestamp)

Step 3: Azure Function (Python)

☐ HTTP-triggered function that receives both files

☐ Match on GSTIN + invoice number + amount, with a tolerance for rounding and a fuzzy match for typos

☐ Classify: matched, mismatched, missing in books, missing in supplier data

☐ Write results to Azure SQL (or Postgres) and return an Excel report

Step 4: Power Automate flow

☐ Trigger: file created in Incoming

☐ Get file content → call the Function → save the report to Processed

☐ Post a Teams adaptive card with the summary and an approval to the finance lead

☐ On approval, email the report; on rejection, move to Exceptions with comments

☐ Error scope: notify on failure and log to the SharePoint list

Step 5: Report and publish

☐ Power BI page on reconciliation status and ageing of mismatches

☐ README with architecture diagram, flow screenshots (export the flow definition), Function code, walkthrough video

Resume line: *Automated GST purchase reconciliation using SharePoint, Power Automate and a Python Azure Function, with fuzzy matching, Teams approvals and Power BI exception tracking.*

Gate B · Apply from week 7

Titles: Intelligent Automation Engineer, Power Platform Developer, Process Automation Engineer, Automation Consultant

Phase 2 · Engineering foundations (weeks 8–11)

Learn

- ☐ FastAPI: routes, path/query params, request bodies, Pydantic models, status codes, dependency injection
- ☐ Background tasks; file uploads
- ☐ Webhooks, pagination, rate limits, retries with backoff, idempotency keys
- ☐ Auth: OAuth2 password flow, JWT access tokens, role-based access
- ☐ Docker: Dockerfile, images, containers, volumes, env vars; docker compose with Postgres
- ☐ GitHub Actions: run Ruff and pytest on push; build the Docker image
- ☐ Azure: Container Apps (or App Service), Key Vault, Application Insights

Project 2 · Payroll Bank Mandate Validation API

Pre-payroll checks that catch bad bank details before salaries go out. Synthetic employees only.

Step 1: Synthetic data

- ☐ Employee master and monthly payroll files with Faker (fake names, fake account numbers)
- ☐ Inject: invalid IFSC format, duplicate account across employees, name mismatch vs master, salary jump vs last month, missing PAN

Step 2: Database

- ☐ Tables: employees, payroll_runs, files, validation_results, exceptions, users, audit_logs

Step 3: API (FastAPI)

- ☐ POST /payroll-files: upload, store, start validation as a background job
- ☐ GET /jobs/{id}: status and counts
- ☐ GET /jobs/{id}/exceptions: paginated, filterable by rule

☐ POST /exceptions/{id}/resolve: mark resolved with a comment (audit logged)

Step 4: Validation rules

☐ IFSC regex, account number length, duplicates, fuzzy name match vs master, salary outlier vs previous month, mandatory fields

☐ Each rule a small tested function; rules configurable in YAML

Step 5: Security

☐ JWT login; roles: uploader, reviewer, admin

☐ Mask account numbers in logs and API responses

Step 6: Ship it

☐ docker compose (api + db); pytest for rules and endpoints

☐ GitHub Actions CI; deploy to Azure; secrets in Key Vault; logs in Application Insights

☐ Auto-generated API docs (/docs) screenshot in README, plus walkthrough video

Resume line: *Built and deployed a containerised FastAPI service on Azure that validates payroll bank mandates with configurable rules, JWT role-based access, audit logging and CI/CD.*

Gate C · Apply from week 11

Titles: Python Automation Engineer, Integration Engineer, Backend Engineer (data), Solutions Engineer (data)

☐ Start Python coding practice: 3–4 medium problems a week

Phase 3 · Modern data stack, properly (weeks 12–14)

You vibe coded dlt + dbt + Dagster before. This time, build it by hand so you can explain every line.

Learn

☐ dlt: sources, resources, incremental loading, schema evolution, write dispositions (append, merge)

- ☐ dbt: models, ref(), sources, materializations (view, table, incremental), tests, docs, seeds, macros basics
- ☐ Medallion layers: bronze (raw), silver (clean), gold (business-ready)
- ☐ Dagster: assets, jobs, schedules, sensors, dbt and dlt integrations

Project 3 · Sales & Inventory Analytics Warehouse

Step 1: Sources

- ☐ Synthetic ERP exports (orders, products, price lists, promotions, inventory movements) as daily CSV drops
- ☐ One REST API source (a public API, or a small FastAPI mock you write) for products or prices

Step 2: Ingestion (dlt → bronze)

- ☐ Incremental loads from files and the API into Postgres bronze schema
- ☐ Handle a schema change (new column) and show how dlt evolves it

Step 3: Transformations (dbt → silver and gold)

- ☐ Silver: typed, deduplicated, standardised staging models
- ☐ Gold: price-volume-mix analysis, slow-moving inventory with ageing buckets, promotion uplift, SKU and channel performance
- ☐ dbt tests (not_null, unique, relationships, accepted_values) and dbt docs

Step 4: Orchestration (Dagster)

- ☐ Assets for dlt and dbt; daily schedule; sensor that runs when a new file lands
- ☐ Failure alert (email or Teams webhook)

Step 5: Serve and publish

- ☐ Dashboard on gold views (your React dashboard or Power BI)
- ☐ docker compose for the whole stack; README with lineage graph from dbt docs; walkthrough video

Resume line: *Designed a medallion data warehouse using dlt, dbt and Dagster on PostgreSQL, with incremental ingestion, tested gold marts for price-volume-mix and slow-moving inventory, and a dashboard layer.*

Gate C+ · Apply from week 14

Titles: Analytics Engineer, Data Engineer, Data Platform Engineer (junior/mid)

Phase 4 · AI engineering (weeks 15–19)

Learn

- ☐ LLM basics: tokens, context windows, temperature, system vs user messages, model choice, cost
- ☐ Structured outputs with JSON schema / Pydantic
- ☐ Tool (function) calling
- ☐ Azure OpenAI as well as a direct provider API
- ☐ Embeddings, chunking, retrieval, reranking (concept), citations; one vector DB: Qdrant
- ☐ Evals: test sets, accuracy, latency, cost; regression testing prompts
- ☐ Guardrails: input/output validation, prompt injection, PII handling, read-only tools, human-in-the-loop

Project 4 · Invoice-to-Ledger AI + Ask-Your-Finance-Data

Step 1: Synthetic invoices

- ☐ Generate 50 PDF invoices with reportlab in 5 different layouts (some scanned-looking)

Step 2: Extraction

- ☐ Pydantic Invoice schema (supplier, GSTIN, invoice number, date, line items, tax, total)
- ☐ LLM structured extraction; validation: totals = lines + tax, GSTIN format, date sanity
- ☐ Low-confidence or failed validation → human review queue (table + simple UI or Excel)
- ☐ Approved invoices load into the Project 0 database

Step 3: Ask-Your-Finance-Data agent

- ☐ Tools: run_readonly_sql over gold views only, get_variance(entity, month), explain_kpi(name)

☐ Guardrails: allowed tables, row limits, no writes, refuse out-of-scope questions

Step 4: RAG over policies

- ☐ Write 10 synthetic accounting SOP / policy documents
- ☐ Chunk, embed, store in Qdrant; answers with citations; "I don't know" when not found

Step 5: Evals

- ☐ Extraction accuracy per field on all 50 invoices
- ☐ 50-question eval set for the agent and RAG with expected answers; track accuracy, latency, cost per question

Step 6: Publish

- ☐ FastAPI endpoints + simple chat UI; Docker; README with eval results table; walkthrough video

Resume line: *Built an LLM invoice-extraction pipeline with schema validation and human review, plus a tool-calling finance Q&A agent and RAG over policies, measured with field-level and question-level evals.*

Gate D · Apply from week 19

Titles: AI Automation Engineer, Applied AI Engineer, GenAI Engineer, AI Application Engineer, AI Integration Engineer

- ☐ If remote traction is weak by now, add relocation roles

Phase 5 · Agents and production (weeks 20–24)

Learn

- ☐ LangGraph: state, nodes, edges, conditional routing, persistence, human-approval interrupts
- ☐ MCP: servers, tools, resources, permissions; connect a client
- ☐ Observability: logs, metrics, traces of every LLM and tool call; cost per run
- ☐ System design: AI document pipeline, support agent, RAG over 10,000 PDFs, safe action-taking agent

Project 5 · AI Data Operations Agent (flagship)

The problem you've seen for years: client files arrive with renamed columns, missing sheets or changed formats, and the automation breaks.

Step 1: Broken-file scenarios

- ☐ Take Project 0 inputs and create 40 broken variants: renamed/missing columns, extra header rows, merged cells, new sheet names, date-format changes, currency codes changed

Step 2: Deterministic first

- ☐ Validator from Project 0 runs first; valid files go straight through

Step 3: Agent for failures (LangGraph)

- ☐ Diagnose node: compare file schema to expected schema, classify the problem
- ☐ Lookup node: RAG over SOPs and past fixes
- ☐ Propose node: column mapping or transformation fix
- ☐ Risk node: auto-fix only low-risk changes; otherwise human approval (Teams via Power Automate, or a webhook)
- ☐ Re-run node: apply fix, re-validate, load, notify

Step 4: Tools via MCP

- ☐ MCP server exposing `read_file_schema`, `validate_file`, `apply_mapping`, `run_pipeline`, `notify`

Step 5: Production

- ☐ FastAPI + Docker + Azure; traces for every step; cost per run; audit log of every fix and approval

Step 6: Evals and publish

- ☐ Run all 40 scenarios: % correctly diagnosed, % safely auto-fixed, % correctly escalated, average cost and time
- ☐ README with architecture diagram, eval table, before/after; 5-minute demo video

Resume line: *Built an AI data-operations agent (LangGraph, MCP, FastAPI, Azure) that diagnoses broken client files, auto-fixes low-risk schema issues and escalates the rest for approval, evaluated on 40 failure scenarios.*

Gate E · Apply from week 24

Titles: Forward Deployed Engineer, Forward Deployed Software Engineer, AI Deployment Engineer, AI Solutions Engineer, Customer Engineer AI

Interview prep

- ☐ 6 STAR stories from client work (no client names): the most complex multi-step automation, an unclear process you uncovered, a production failure, pushing back on a request, measurable impact (2–3 days → 30 minutes), learning a tool fast
 - ☐ System design practice weekly; 2 mock interviews a week
-

Not now (skip until a specific job asks)

- UiPath and Power Automate Desktop
- Airflow (Dagster covers the concept)
- Fabric, Databricks, PySpark, Snowflake
- MLOps courses and ML theory books
- AWS, GCP, multiple certifications (one Azure certification at most; confirm the current exam before booking)
- Kubernetes, Kafka, Terraform, CrewAI, AutoGen, LlamaIndex

Resources (one per topic)

Topic	Resource
SQL practice	LeetCode SQL 50 or DataLemur; PostgreSQL docs
Python + pandas	Official Python tutorial; pandas user guide
Power Automate	Microsoft Learn — Power Automate
FastAPI	FastAPI official tutorial
Docker	Docker Get Started
dlt, dbt, Dagster	Each tool's official docs and tutorials
LLM apps, RAG, evals	LLM Zoomcamp • provider docs
Vector DB	Qdrant tutorials
Agents	LangChain / LangGraph docs
Book	AI Engineering by Chip Huyen

Sources

- [CIEL HR: FDE demand study, Aug 2026](#)
- [BuildFastWithAI: FDE salary bands India 2026](#)
- [FDE Academy: FDE jobs in India](#)
- [Salesforce FDE listing](#)
- [Lenovo FDE listing](#) · [Apple FDE listing](#)